Cognitive Load Estimation from EEG + fNIRS

Improving and honestly evaluating the binary (Low vs High) workload model

TU Berlin n-back dataset · 26 subjects · 4,576 windows

42 controlled experiments · 26-fold leave-one-subject-out + 8-subject lockbox

Run **run_20260820_141126_19cb**

commit 12b29b33c2ce · branch binary-class

torch 2.13.0 · seed 42

generated 20 August 2026

Contents

01 The problem and the data . . . 2
02 Feature representation . . . 3
03 Model architecture . . . 3
04 Evaluation protocol . . . 4
05 Leakage audit . . . 4
06 The experiments . . . 6
07 Final results . . . 9
08 What the experiments found . . . 12
09 Feature importance . . . 14
10 Learned representations . . . 16
11 Limitations . . . 17
12 Reproducing this work . . . 17

01 The problem and the data

This project estimates **mental workload** from two brain signals recorded at the same time. **EEG** measures electrical activity at 200 Hz — fast, noisy, sensitive to cortical rhythms. **fNIRS** measures blood oxygenation at 10 Hz — slow, metabolic, reported as oxygenated (HbO) and deoxygenated (HbR) haemoglobin. They are complementary, and fusing them is the modelling premise.

The data is the open **TU Berlin dataset (Shin et al., 2018)**: 26 subjects performing an n-back working-memory task with 25 trials each. The label is not a subjective rating — it is the experimentally imposed difficulty, which is what makes it usable as ground truth. 0-back is **Low** load, 2-back **Medium**, 3-back **High**.

This work addresses the **binary** framing: Low (0-back) versus High (3-back), dropping the middle class. The result is perfectly balanced — 2,288 windows per class.

Property	Value
Subjects	26
Trials	416 (25 per subject)
EEG	30 channels @ 200 Hz
fNIRS	36 channels @ 10 Hz (HbO and HbR)
Window	10 s, 5 s stride (50% overlap)
Windows (binary)	4,576
Classes	Low (0-back) vs High (3-back)
Chance	0.5000

02 Feature representation

Each window is reduced to **1,476 features**. The column names are *channel-major* — all of channel 00's features, then channel 01's, and so on — which means each modality block reshapes exactly onto a (channel × feature-type) grid. That property is what later makes a convolutional model a legitimate thing to try.

```
EEG    540 = 30 electrodes x 18 feature types
             (6 bands: delta, theta, alpha, beta, gamma, engagement
              x 3 statistics: mean, std, slope)

fNIRS  936 = 36 optodes x 26 feature types
             (13 statistics: mean, std, slope, min, max, ptp, auc, power,
              skew, kurtosis, early, late, delta   x HbO and HbR)
```

The preprocessing contract is `nan_to_num -> per_subject_norm -> log -> scaler`. Per-subject z-scoring removes each person's individual baseline; EEG band powers are right-skewed so they get a log, and HbO/HbR go negative so they get a *signed* log. The scalers — RobustScaler for EEG, StandardScaler for fNIRS — are refit from scratch on each fold's training rows only.

03 Model architecture

Two parallel modality encoders feed a fusion layer and a classification head. The encoder family is swappable, which is what makes the MLP-versus-CNN comparison controlled: everything downstream is held fixed.

```
EEG   (540) --> encoder --+
                          +--> fusion --> FC head --> softmax
fNIRS (936) --> encoder --+
```

Encoder families

MLP (the served model): a projection to a fixed width, then *N* residual MLP blocks, then a projection to the encoder dimension. Residual connections keep gradients healthy in a small-data regime.

1D CNN (added here): reshape onto the channel grid and convolve along the **electrode / optode axis**, with the feature types as the convolution's input channels. One filter bank is shared across all channels — a real inductive bias, and an order-of-magnitude fewer input-stage parameters. It assumes neighbouring channel indices are spatially related, which is only approximately true for this montage, so it was tested rather than assumed.

Fusion

Attention fusion computes a sigmoid gate per modality from the concatenation of both embeddings, then scales each modality by its own gate before fusing. The model learns how much to trust EEG versus fNIRS, and those gate values are what the web app's modality bar displays. **Concat fusion** is the ablation.

Domain-adversarial option (DANN)

A subject discriminator sits behind a gradient-reversal layer on the fused embedding: forward pass is the identity, backward pass multiplies the gradient by $-\lambda$. The discriminator gets better at identifying subjects while the same signal pushes the encoder to *erase* subject identity. Domain labels are the training subjects only, re-indexed per fold, so the discriminator never has a class for the held-out subject.

The selected model has **245,594 parameters**: projection width 96, encoder dimension 48, 2 residual blocks, attention fusion, head [48, 24], dropout 0.5 / 0.6.

04 Evaluation protocol

A 26-fold leave-one-subject-out score is a good generalization estimate — but if you run forty experiments and keep the best one, that score is a selection artifact: the folds have quietly become a training signal. So the subjects are split once, deterministically, into two pools.

```
development pool  18 subjects  [1, 2, 3, 4, 5, 7, 9, 10, 11, 12, 13, 14, 15, 17, 20, 23, 24, 25]
lockbox            8 subjects  [6, 8, 16, 18, 19, 21, 22, 26]
```

Every architecture, hyperparameter, scheduler, dropout rate and dataset variant was chosen on **development-pool LOSO alone**. The lockbox was never read during the search; after the configuration was frozen, each lockbox subject was evaluated once by a model trained only on development subjects. The 26-fold LOSO number is also reported, labelled *selection-informed*, so it stays comparable to the rest of the project.

Inside one fold

```
held-out subject   -->  test only. Never in train, val, or any fitted statistic.
remaining subjects -->  GroupShuffleSplit by subject  -->  train + validation
```

Checkpoints are selected on **validation macro F1** with best-weight restore, on subjects the weights never saw.

> **The selection rule, fixed in advance.** Rank by mean development-LOSO macro F1; take every arm within one standard error of the leader; inside that set prefer the incumbent, then the smallest model. A challenger must clear the noise floor to be adopted. With 18 folds and a fold-to-fold standard deviation near 0.08, that floor is **±0.0184**.

> **A flaw that was caught mid-campaign.** The first implementation broke ties only on parameter count. Arms within a stage usually have *identical* parameter counts, so the rule silently degenerated into "pick the highest mean" — noise-chasing. It surfaced when a stage accepted a +0.0023 improvement against a ±0.018 floor. Over ten stages that compounds into a selection artifact, so the rule was corrected and the entire campaign was restarted. The discarded run is still in the archive.

05 Leakage audit

Nine assertions run before *every* fold trains. Any failure raises an exception and aborts the run, so a leaked score is never produced and therefore can never be reported.

Check	What it proves
`row_index_disjoint`	no row appears in two splits
`subject_disjoint`	train / val / test share no subject
`trial_disjoint`	no trial straddles a split
`window_identity_disjoint`	no (subject, trial, window) key on both sides
`duplicate_feature_rows`	no byte-identical feature row across splits
`scaler_fit_on_train_only`	rows used to fit the scalers are a subset of training rows
`heldout_subject_never_fitted`	the held-out subject is absent from train and val
`no_nan_or_inf`	preprocessing produced finite values
`labels_valid`	labels in range, training fold has both classes

The sliding-window question

Adjacent windows overlap by 50%, so they share raw signal — putting two of them on opposite sides of a split would leak. Grouping by subject makes that structurally impossible: every window of the held-out subject is in the test set and none is anywhere else. The audit asserts it explicitly rather than relying on the argument.

Per-subject normalization — the honest caveat

Each subject is z-scored against their own statistics, and that happens before the fold split. It **cannot** leak between subjects, and this is proved numerically rather than in prose: the whole preprocessing chain is re-run for one subject *from that subject's raw rows in isolation* and checked against the full-dataset computation. Observed difference: **exactly zero**.

It is, however, *transductive within a subject* — computing a mean needs the whole session. That is standard practice in BCI and uses no labels, but it means deployment needs a recorded session, not a single window. The cost of restricting it to a short calibration block was measured (see section 08).

> **The audit is not vacuously true.** A test suite injects each leakage type deliberately — a subject leak, a window leak, a scaler-fit leak, a duplicated row, a NaN — and asserts the audit catches that specific check. All 8 pass. A separate suite asserts the protocol machinery, including that early stopping restores weights which reproduce the best validation score to 1e-6. All 6 pass.

06 The experiments

42 arms across 10 stages. Each stage varied one axis from the incumbent and the winner carried forward. All arms trained and evaluated on identical development folds, so every comparison is paired.

Stage	Axis varied	Best challenger	Δ	Outcome
`S1_batch_lr`	batch size, learning rate	`S1_bs64_lr1e3`	-0.0170	rejected
`S2_schedule`	epoch budget, LR schedule	`S2_no_sched`	+0.0022	rejected (inside noise)
`S3_weight_decay`	weight decay	`S3_wd0`	-0.0099	rejected
`S4_dropout`	dropout rates	`S4_drop_low`	+0.0085	rejected (inside noise)
`S5_capacity`	width, depth	`S5_proj192`	+0.0160	rejected (inside noise)
`S6_architecture`	MLP vs CNN, fusion	`S6_mlp_concat`	-0.0157	rejected
`S7_modality_dropout`	modality dropout	`S7_md010`	-0.0055	rejected
`S8_noise`	input noise, label smoothing	`S8_noise20`	-0.0072	rejected
`S9_dann`	domain-adversarial λ	`S9_dann01`	-0.0270	rejected
`S10_data`	window geometry, normalization, HbR	`S10_stride1s`	+0.0115	rejected (inside noise)

> **No stage produced a change that cleared the noise floor.** The configuration the project had already locked in survived every axis tested.

Full leaderboard — 18-fold development LOSO

Δ and *p* are paired against the baseline on the same folds. *gap* is train − held-out macro F1: higher means more overfitting.

Experiment	Macro F1	Acc	AUC	Δ	p	gap	Params	Description
S5_proj192	0.6611 ± 0.101	0.669	0.721	+0.0160	0.309	0.146	620,570	projection width 192
S10_stride1s	0.6566 ± 0.114	0.662	0.721	+0.0115	0.461	0.217	245,594	dense windows: 10s / 1s stride (4.6x rows)
S4_drop_low	0.6537 ± 0.120	0.658	0.710	+0.0085	0.655	0.194	245,594	dropout 0.3 / 0.3
S2_no_sched	0.6474 ± 0.093	0.655	0.716	+0.0022	0.819	0.233	245,594	no LR schedule (control)
S5_proj32_enc32	0.6466 ± 0.112	0.656	0.726	+0.0015	0.929	0.152	66,114	narrow: proj 32 / enc 32 / head [32]
A0_baseline	**0.6451 ± 0.078**	**0.654**	**0.717**	**baseline**		**0.163**	**245,594**	**served binary architecture + config.py training hyperparameters**
S5_blocks1	0.6433 ± 0.084	0.647	0.704	-0.0018	0.890	0.158	207,578	1 residual block
S4_drop_mid	0.6414 ± 0.104	0.645	0.697	-0.0037	0.804	0.245	245,594	dropout 0.4 / 0.5
S7_md010	0.6396 ± 0.100	0.648	0.710	-0.0055	0.688	0.189	245,594	modality dropout 0.1 / 0.1
S8_noise20	0.6380 ± 0.105	0.647	0.713	-0.0072	0.635	0.184	245,594	input noise 0.20
S3_wd0	0.6352 ± 0.110	0.651	0.712	-0.0099	0.494	0.144	245,594	no weight decay
S2_longer_p25	0.6316 ± 0.100	0.642	0.702	-0.0136	0.107	0.214	245,594	longer budget, patience 25
S2_longer_p15	0.6316 ± 0.100	0.642	0.702	-0.0136	0.107	0.214	245,594	longer budget, patience 15
S6_mlp_concat	0.6294 ± 0.101	0.636	0.683	-0.0157	0.230	0.157	236,282	MLP encoders, concat fusion (attention ablation)
S2_cosine	0.6294 ± 0.113	0.638	0.701	-0.0157	0.270	0.254	245,594	cosine annealing
S3_wd01	0.6287 ± 0.107	0.640	0.709	-0.0164	0.140	0.207	245,594	weight decay 0.1
S3_wd003	0.6283 ± 0.098	0.637	0.701	-0.0168	0.296	0.166	245,594	weight decay 0.003
S1_bs64_lr1e3	0.6281 ± 0.122	0.634	0.694	-0.0170	0.292	0.236	245,594	batch 64, lr 1e-3
S8_noise10	0.6187 ± 0.097	0.628	0.683	-0.0264	0.094	0.151	245,594	input noise 0.10
S9_dann01	0.6182 ± 0.098	0.626	0.680	-0.0270	0.104	0.160	245,594	DANN lambda 0.1
S10_win8s	0.6160 ± 0.105	0.622	0.677	-0.0291	0.128	0.172	245,594	8s windows / 4s stride

S7_md_eeg_only	0.6150 ± 0.091	0.625	0.676	-0.0301	0.172	0.175	245,594	drop EEG more often (0.3 / 0.1)
S10_norm_after_log	0.6148 ± 0.099	0.624	0.684	-0.0303	**0.050**	0.173	245,594	per-subject norm applied after the log
S7_md020	0.6110 ± 0.117	0.628	0.697	-0.0341	0.209	0.157	245,594	modality dropout 0.2 / 0.2
S5_proj48	0.6061 ± 0.106	0.610	0.654	-0.0390	0.144	0.118	113,402	projection width 48
S10_calibration_norm	0.6049 ± 0.114	0.624	0.671	-0.0402	0.156	0.185	245,594	per-subject norm from 30 calibration windows only
S8_smooth	0.6048 ± 0.141	0.626	0.678	-0.0403	0.171	0.183	245,594	label smoothing 0.1
S1_bs64	0.5978 ± 0.120	0.616	0.677	-0.0473	**0.019**	0.115	245,594	batch 64
S9_dann05	0.5968 ± 0.117	0.613	0.668	-0.0483	**0.014**	0.156	245,594	DANN lambda 0.5
S1_bs128	0.5864 ± 0.104	0.607	0.688	-0.0587	**0.009**	0.113	245,594	batch 128
S3_wd005	0.5846 ± 0.116	0.601	0.661	-0.0605	**0.004**	0.222	245,594	weight decay 0.05
S5_blocks3	0.5841 ± 0.100	0.606	0.652	-0.0610	**0.011**	0.135	283,610	3 residual blocks
S8_noise0	0.5831 ± 0.094	0.601	0.652	-0.0620	**0.025**	0.134	245,594	no input noise
S10_no_hbr	0.5781 ± 0.098	0.592	0.635	-0.0670	**0.014**	0.086	200,666	HbO only (drop HbR)
S9_dann10	0.5778 ± 0.110	0.598	0.663	-0.0674	**0.012**	0.125	245,594	DANN lambda 1.0
S10_no_subject_norm	0.5698 ± 0.106	0.589	0.635	-0.0753	**0.014**	0.125	245,594	no per-subject normalization
S6_cnn_res	0.5550 ± 0.084	0.576	0.604	-0.0902	**0.002**	0.026	107,962	CNN + residual blocks, concat fusion
S1_lr1e4	0.5547 ± 0.100	0.590	0.646	-0.0904	**0.004**	0.063	245,594	lr 1e-4
S6_cnn_res_attn	0.5467 ± 0.079	0.560	0.588	-0.0984	**0.001**	0.035	117,274	CNN + residual + attention fusion
S6_cnn_wide	0.5467 ± 0.071	0.566	0.605	-0.0984	**0.000**	0.076	363,930	CNN + residual + attention, 128 filters
S4_drop_high	0.5083 ± 0.127	0.565	0.602	-0.1369	**0.000**	0.040	245,594	dropout 0.6 / 0.7
S6_cnn	0.4786 ± 0.087	0.531	0.552	-0.1665	**0.000**	0.026	54,842	1D CNN over channel axis, concat fusion

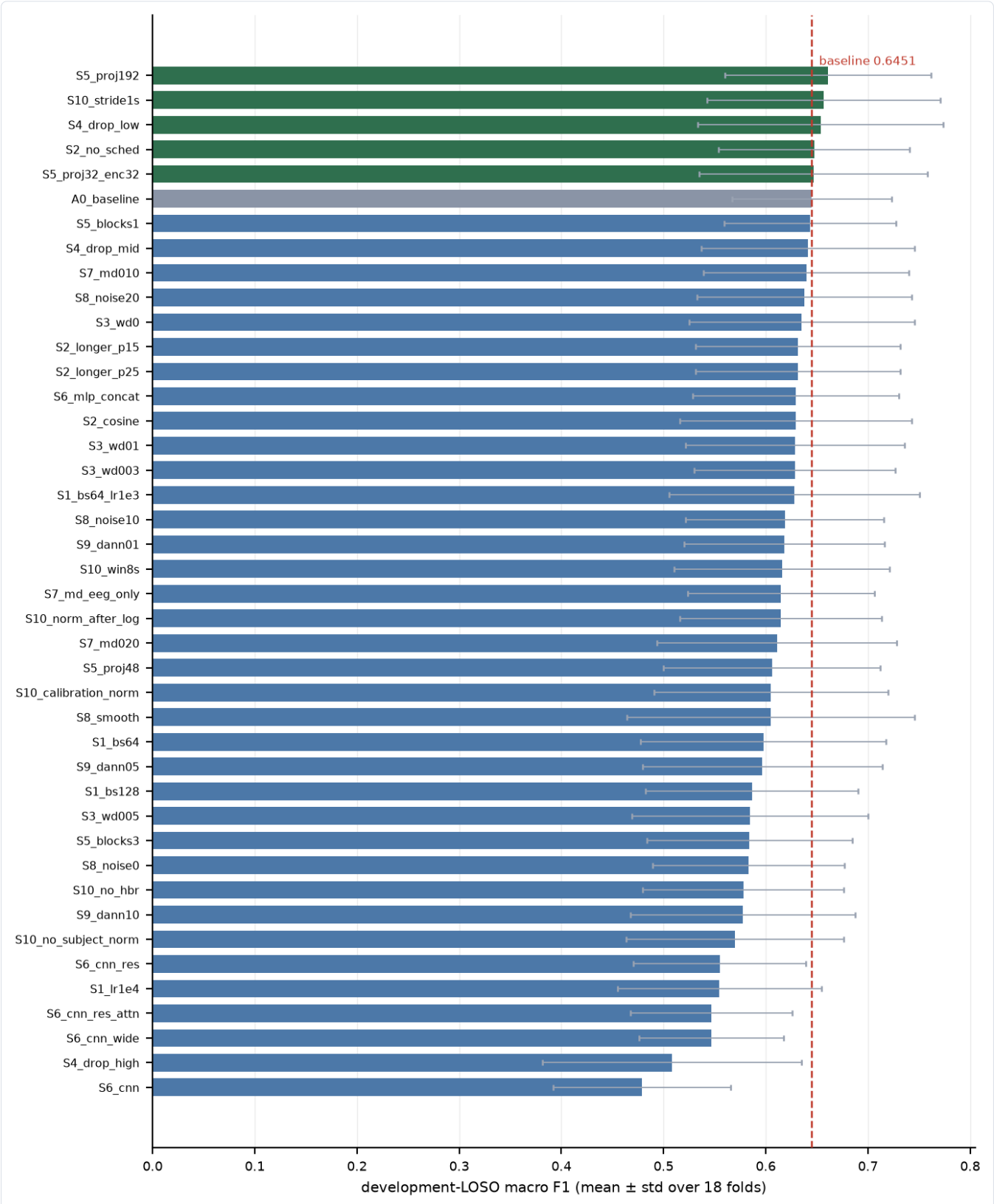


Figure 1. All 42 arms across both campaign runs, ranked by development-LOSO macro F1. Grey is the baseline, green beat it, blue did not; the dashed line is the baseline score. Error bars are the standard deviation across the 18 development folds — note how much wider they are than the gaps between arms.

07 Final results

The selected model is `A0_baseline` — the configuration already being served. Nothing beat it.

0.6309	0.6255	0.6371	0.6275
LOSO ACCURACY	LOSO MACRO F1	LOCKBOX ACCURACY	LOCKBOX MACRO F1

Metric	26-fold LOSO (all subjects)	Lockbox (8 unseen subjects)
Accuracy	0.6309 ± 0.101	0.6371 ± 0.085
Macro F1	0.6255 ± 0.104	0.6275 ± 0.093
Balanced accuracy	0.6309 ± 0.101	0.6371 ± 0.085
ROC-AUC	0.6775 ± 0.129	0.6888 ± 0.111
Matthews correlation	0.2675 ± 0.206	0.2816 ± 0.169
Cohen's Kappa	0.2618 ± 0.202	0.2741 ± 0.170

> **The lockbox number is not lower than the LOSO number.** That is the check the whole protocol existed for: forty-two experiments of searching produced **zero selection inflation**. The 63% is real.

Training, validation and testing accuracy

"Testing" here means **a person the model has never seen**. There is no random train/test split anywhere in this work. A random window split would score considerably higher, but it would be measuring the model's ability to recognise a subject's physiological baseline, not workload.

Protocol	Train	Validation	Test (held-out subject)	Gap
26-fold LOSO	0.8022 ± 0.093	0.6400 ± 0.053	**0.6309 ± 0.101**	+0.171
Lockbox	0.8186 ± 0.059	0.6783 ± 0.038	**0.6371 ± 0.085**	+0.182

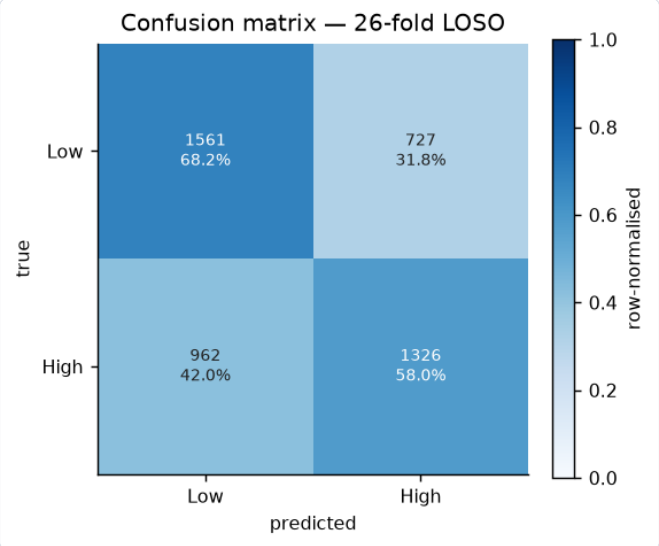


Figure 2. Pooled over all 4,576 held-out windows from the 26 folds.

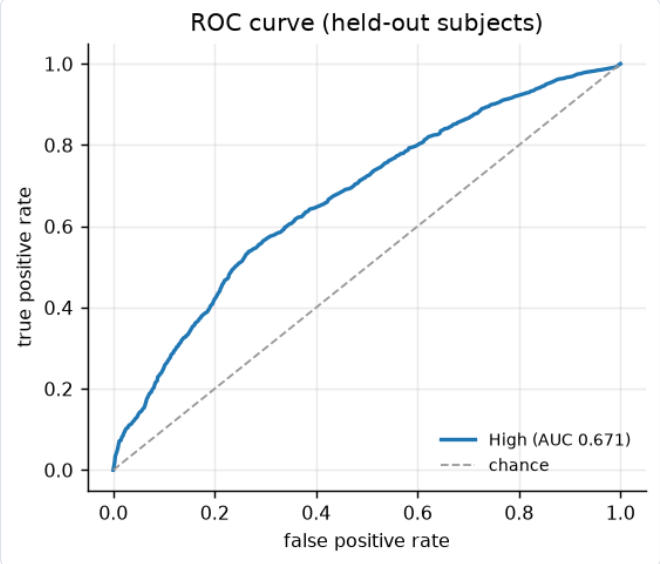


Figure 3. ROC across every fold's held-out probabilities. AUC 0.6707.

Class	Precision	Recall	F1	Support
Low	0.619	0.682	0.649	2,288
High	0.646	0.580	0.611	2,288

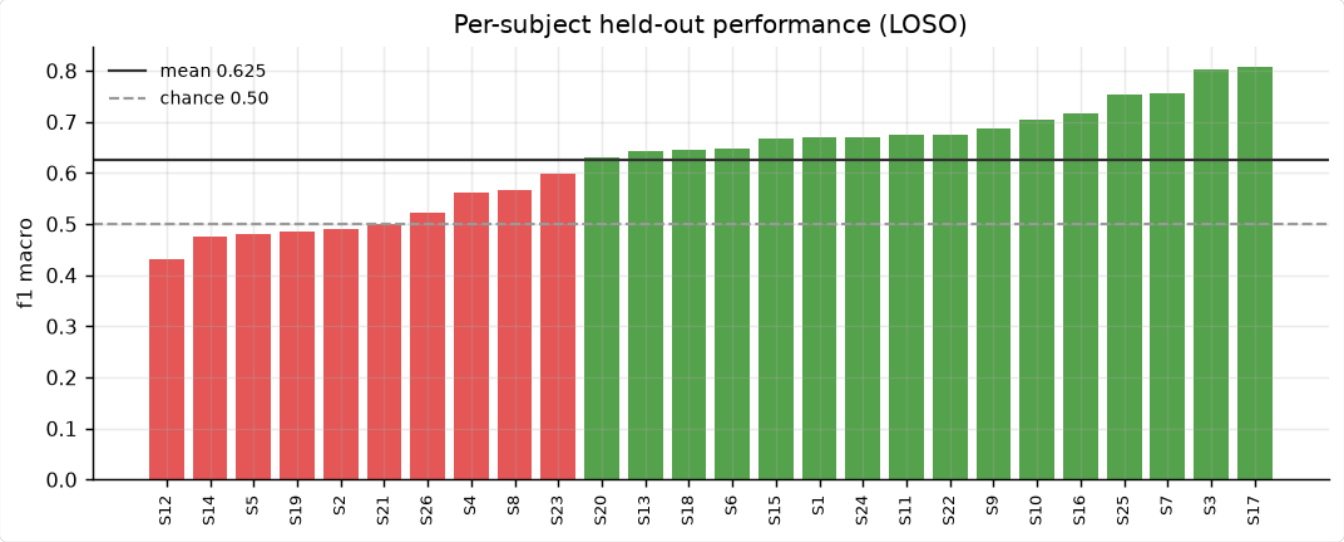


Figure 4. Macro F1 for each held-out subject. Every bar is one person the model never trained on.

The dominant source of error is which person you test on, not which model you chose. The best subject scores 0.807 and the worst 0.431 — a spread of 0.38. That is more than twice the largest difference between any two of the 42 models tested. 4 of 26 subjects fall below chance accuracy.

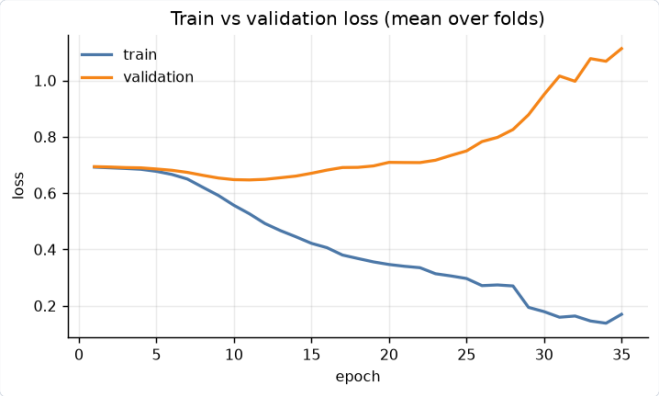


Figure 5. Train versus validation loss, averaged over folds.

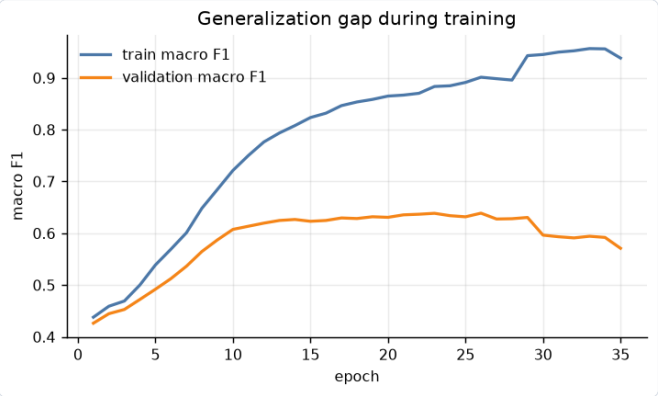


Figure 6. The macro-F1 gap opening during training.

08 What the experiments found

The CNN does not help this representation

Architecture	Macro F1	Δ	p	gap	Params
A0_baseline	**0.6451**	**baseline**		**0.163**	**245,594**
S6_cnn_res	0.5550	-0.0902	0.002	0.026	107,962
S6_cnn_res_attn	0.5467	-0.0984	0.001	0.035	117,274
S6_cnn_wide	0.5467	-0.0984	0.000	0.076	363,930
S6_cnn	0.4786	-0.1665	0.000	0.026	54,842

Every variant is significantly worse. The interesting column is *gap*: the CNNs cut overfitting from 0.163 to 0.026–0.076 — sharing one filter bank across electrodes nearly **eliminates** it. But it removes so much capacity that held-out performance falls further than the overfitting it cures.

The interpretation matters more than the verdict. These are *pre-aggregated per-channel statistics*, not raw time series. A convolution along the channel axis of 18 summary numbers per electrode has no local structure to exploit. A CNN would be the right tool on the raw signal; on this representation the spatial prior costs more than it buys.

Regularization cannot close the gap

Weight decay (0 to 0.1), dropout (0.3 to 0.7), input noise (0 to 0.2), label smoothing and modality dropout were all tested. None improved held-out performance beyond noise, and the pattern is one-directional: stronger regularization reliably reduces the gap *and* reduces held-out performance with it. Dropout 0.6 / 0.7 cuts the gap from 0.163 to 0.040 while losing 0.137 macro F1 ($p < 0.001$).

The gap is not excess capacity that can be regularized away. It reflects genuine between-subject variability: the model fits its training subjects well, and those subjects are simply not fully representative of the next person.

Capacity is not the constraint in either direction

A model with **66,114 parameters** reaches 0.6466 — statistically identical to the 245,594-parameter baseline (Δ +0.0015, $p = 0.929$) at **3.7× smaller**. Widening to 620,570 parameters gains only +0.016, inside the noise floor. If model size matters for your deployment, the small model is free.

Domain-adversarial training is rejected — and the probe explains why

λ	Macro F1	Δ	p
0 (baseline)	**0.6451**	—	—
0.1	0.6182	-0.0270	0.104
0.5	0.5968	-0.0483	0.014
1.0	0.5778	-0.0674	0.012

Performance degrades **monotonically** with adversarial strength, independently replicating the project's earlier 3-class finding on the binary task under a stricter protocol. The subject-invariance probe — a fresh classifier trained to recover subject identity from each representation — explains the mechanism.

Representation	Probe accuracy	Chance
raw features	0.013	0.125
baseline model	0.214	0.125
selected model	0.214	0.125
dann model	0.217	0.125

Per-subject z-scoring has already stripped subject identity from the *raw* features. The encoder then **re-introduces** it, and DANN **fails to remove it** — no better than the baseline it was meant to improve. The adversary injects noise into the task head without deleting the confound it targets. That is a coherent negative result, not an inconclusive one.

Four existing design choices, confirmed and priced

Variant	Δ macro F1	p	Reading
`no per-subject normalization`	-0.0753	**0.014**	per-subject normalization genuinely matters
`HbO only (drop HbR)`	-0.0670	**0.014**	HbR genuinely helps
`per-subject norm applied after the log`	-0.0303	**0.050**	normalizing before the log was the right call
`per-subject norm from 30 calibration w`	-0.0402	0.156	the deployment-realistic mode costs ~0.04, not significant
`dense windows: 10s / 1s stride (4.6x r`	+0.0115	0.461	denser windows buy nothing measurable
`8s windows / 4s stride`	-0.0291	0.128	10 s windows beat 8 s

The **calibration-only** result is the practically useful one: restricting per-subject statistics to the first 30 windows — a short calibration block rather than the whole session — costs about 0.04 macro F1 and is not statistically significant. That makes an online deployment defensible.

09 Feature importance

0.945	0.055	0.458	0.488
FNIRS SHARE	EEG SHARE	HBO SHARE	HBR SHARE

Computed with SHAP's gradient explainer on this run's own trained model over held-out windows. **fNIRS dominates overwhelmingly** — far more starkly than the original project's near-parity attention gates suggested. And **HbR slightly exceeds HbO**, which independently corroborates the ablation showing that removing HbR costs 0.067 macro F1.

#	Feature family	Modality	Total mean |SHAP|
1	HBR auc	fNIRS	0.3016
2	HBO max	fNIRS	0.2824
3	HBO min	fNIRS	0.2716
4	HBO mean	fNIRS	0.2695
5	HBR min	fNIRS	0.2592
6	HBR power	fNIRS	0.2503
7	HBO late	fNIRS	0.2442
8	HBR max	fNIRS	0.2412
9	HBO early	fNIRS	0.2409
10	HBR late	fNIRS	0.2362

The dominant signal is haemodynamic response *shape* — area under the curve, maximum, minimum, mean, the late-window response — not EEG band power.

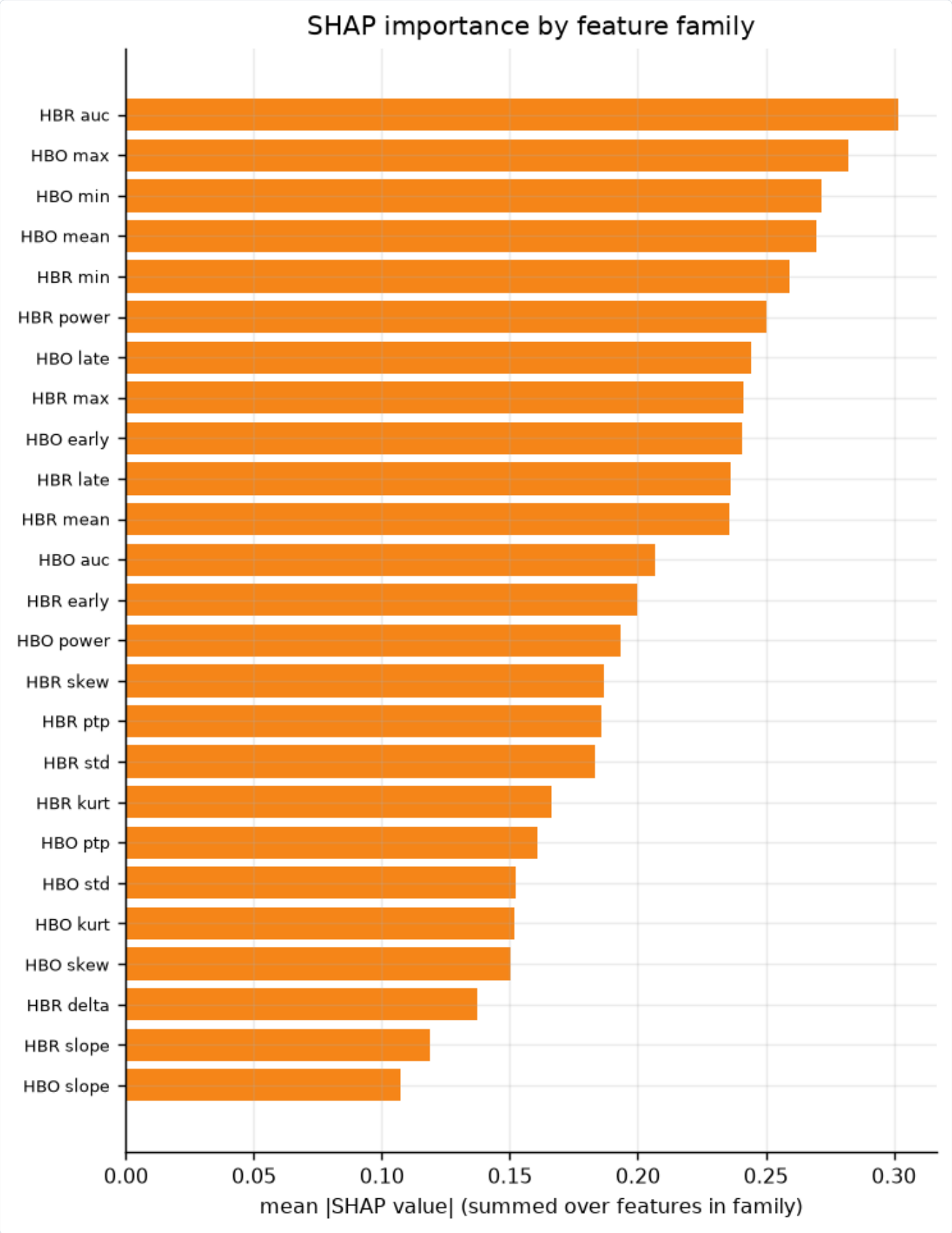


Figure 7. Attribution mass by feature family. Blue is EEG, orange fNIRS.

10 Learned representations

Every variant is trained on the same development subjects and embedded on the same unseen subjects, so the pictures differ only by the model that produced them.

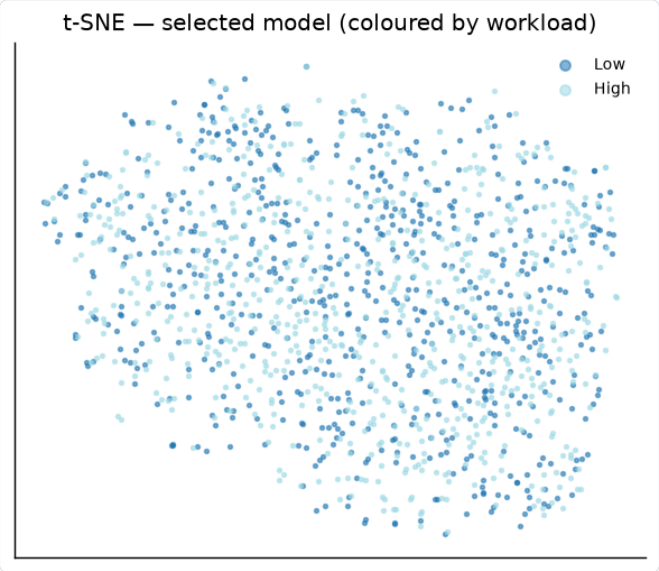


Figure 8. t-SNE of the fused embedding, coloured by workload class.

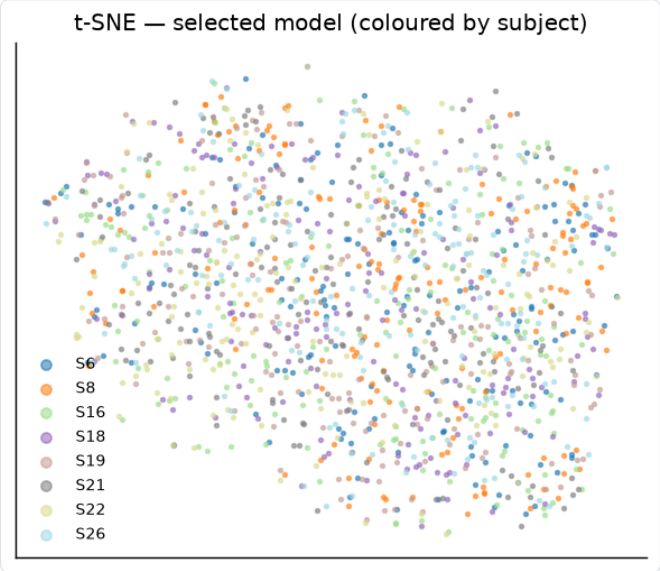


Figure 9. The same embedding, coloured by subject. Subject clustering is visible — which is the ceiling.

11 Limitations

1. **Statistical power.** With 18 development subjects the standard error on mean macro F1 is ±0.018, so this search could only reliably detect improvements larger than roughly 0.04. A smaller genuine gain would be invisible to it.
2. **26 subjects is small** for cross-subject claims, and 4 of them fall below chance. The between-subject spread dominates everything else measured here.
3. **Per-subject normalization is transductive.** It uses no labels and no other subject's data, but it needs the subject's whole session. The calibration mode prices that at about 0.04 macro F1.
4. **The greedy staged search is not exhaustive.** It varies one axis at a time from a moving incumbent, so it cannot find improvements requiring two simultaneous changes.
5. **The 26-fold LOSO number is selection-informed** — the configuration was chosen using 18 of the 26 subjects it evaluates on. The lockbox number is the clean estimate, and it agrees.
6. **SHAP is one attribution method on one trained model.** The fNIRS dominance is much starker than the original attention-gate analysis suggested; the HbR ablation corroborates the direction but not the magnitude.
7. **The CNN result is specific to this feature representation.** It says nothing about a CNN over the raw time series, which was never tested.

12 Reproducing this work

```
# Full campaign (several hours on CPU)
.venv/bin/python run_campaign.py --tag my-run

# A single stage, or the 3-class task
.venv/bin/python run_campaign.py --stages S6
.venv/bin/python run_campaign.py --mode 3class

# Smoke test: reduced cohort, every code path in ~2 minutes
.venv/bin/python run_campaign.py --quick --limit-subjects 7 --max-epochs 6

# Tests and independent verification
.venv/bin/python -m research.test_leakage      # 8/8
.venv/bin/python -m research.test_framework    # 6/6
.venv/bin/python -m research.verify_run --all  # recompute metrics from predictions

# The app
cd backend && ../.venv/bin/python -m uvicorn main:app --port 8000
cd frontend && npm run dev            # http://localhost:5173
```

Every run writes its own directory under `outputs/runs/`, created before any data is read, so a run that crashes still leaves its metadata, config and traceback behind. Nothing is ever overwritten and failed runs are never deleted. One row per evaluated window is saved to `predictions.csv`, and every metric in the archive can be recomputed from it — `research.verify_run` does exactly that and reports zero mismatches.

Further reading

- `explanation/PROJECT_RECORD.md` — the complete record, in full detail
- `explanation/BINARY_MODEL_IMPROVEMENT.md` — framework methodology
- `explanation/API_AND_ARCHITECTURE.md` — endpoint-by-endpoint API reference
- `explanation/PROJECT_OVERVIEW.md` — the original research project
- `RESULTS_BINARY_IMPROVEMENT.md` — machine-generated results tables

Conclusion

The binary model was not improved, and that is the result. Across 42 controlled experiments spanning batch size, learning rate, LR schedule, epoch budget, weight decay, dropout, capacity, depth, architecture family, fusion type, modality dropout, input noise, label smoothing, domain-adversarial training, window geometry, normalization mode and HbR inclusion, nothing beat the served configuration by more than the noise floor, and most alternatives were significantly worse.

The configuration the project had already locked in sits at a local optimum for this feature representation. Three mutually consistent lines of evidence say the remaining error is not a modelling deficiency: regularization trades the gap for performance one-for-one; the subject probe shows the encoder re-introduces subject identity that preprocessing had removed and DANN cannot remove it again; and the between-subject spread is more than twice the largest difference between any two models tested.

> The final honest number is **63.1% accuracy on people the model has never seen** (chance 50%), and the lockbox confirms it is not inflated by the search. The most promising untested direction is not a better classifier — it is either more subjects, or a genuinely different feature representation where an architectural prior like convolution would have something to work with.